অধ্যায় ১৬

JavaScript Error Handling, Debugging, Testing এবং Software Reliability

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Software Reliability-র চারটি দিক—সঠিকতা, সহনশীলতা, পুনরুদ্ধার এবং পর্যবেক্ষণযোগ্যতা—ব্যাখ্যা করতে।
- `code`, `retryable` ও `cause`-সহ একটি Error-এর শ্রেণিবিন্যাস তৈরি করতে, এবং `AggregateError` সামলাতে।
- Invariant ও Assertion দিয়ে Program-এর ভেতরের নিয়ম পাহারা দিতে।
- Structured Log লিখতে, এবং একটি পুনরাবৃত্তিযোগ্য Debugging পদ্ধতি অনুসরণ করতে।
- Test-এর ধরন (Unit, Integration, End-to-End, Regression, Boundary) এবং Testing Pyramid বুঝতে।
- Node.js-এর `node:test` গভীরভাবে ব্যবহার করতে: `describe`, Hook, Subtest, Mock, Coverage।
- Stub, Spy, Mock ও Fake দিয়ে বাইরের নির্ভরতা নিয়ন্ত্রণ করে নিশ্চিত Test লিখতে।
- সাময়িক ত্রুটিতে Retry, ব্যর্থতায় Compensation, এবং দুবার পাঠানো অনুরোধে Idempotency নিশ্চিত করতে।
- একটি সম্পূর্ণ **BookingGuard** Booking Service তৈরি ও বহুস্তরে পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ১০: Promise, `async` / `await`, Retry ও Backoff।
- অধ্যায় ১১: `try` / `catch` / `finally`, নিজের Error, `cause`, প্রত্যাশিত বনাম অপ্রত্যাশিত ত্রুটি।
- অধ্যায় ১২–১৪: ES Module, Class, Dependency Injection, Fake Timer।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ১১-এ আপনি একটি ত্রুটি ঘটলে কী করবেন তা শিখেছেন। এই অধ্যায়ের প্রশ্ন বড়: একটি Program কীভাবে **নির্ভরযোগ্য** হয়? একটি Booking Service-এর কথা ভাবুন। Payment Gateway মাঝে মাঝে সাময়িক ত্রুটি দেয়; ব্যবহারকারী অধৈর্য হয়ে Button দুবার চাপেন; Email Server বন্ধ থাকে; শেষ আসনটির জন্য দুজন একসঙ্গে চেষ্টা করেন। প্রতিটি পরিস্থিতিতে Program-কে সঠিক থাকতে হবে: দুবার টাকা কাটা যাবে না, আসন হারাবে না, এবং Email না গেলেও Booking ব্যর্থ হবে না।

এই ধরনের আচরণ শুধু সতর্কভাবে Code লিখে নিশ্চিত হয় না; প্রমাণ করতে হয়। প্রমাণের হাতিয়ার Automated Test, এবং এই অধ্যায়ের অর্ধেক Test নিয়ে: কী Test করবেন, কীভাবে বাইরের জগৎকে (Payment, Database, সময়) Test-এ নিয়ন্ত্রণে আনবেন, এবং কীভাবে একটি Bug আর কখনো ফিরে না আসে তা নিশ্চিত করবেন।

১৬.১ Software Reliability কী

দিক	প্রশ্ন	উদাহরণ
সঠিকতা (Correctness)	স্বাভাবিক অবস্থায় ফল কি ঠিক?	পাঁচ আসনে গ্রুপ ছাড় ঠিকমতো হিসাব
সহনশীলতা (Robustness)	অস্বাভাবিক ইনপুট বা পরিবেশে কি ভেঙে পড়ে?	ভুল Email, Payment-এর সাময়িক ত্রুটি
পুনরুদ্ধার (Recoverability)	ব্যর্থতার পর অবস্থা কি সঠিক থাকে?	Payment ব্যর্থ হলে সংরক্ষিত আসন ফেরত
পর্যবেক্ষণযোগ্যতা (Observability)	সমস্যা হলে কি জানা যায় কী হয়েছে?	প্রতিটি Retry ও ব্যর্থতার Log

অধ্যায় ১১-এর মূল নিয়মগুলো এখানে ভিত্তি: প্রত্যাশিত ত্রুটি নিজের ধরনে এবং কাছে ধরা; অপ্রত্যাশিত ত্রুটি ওপরে যেতে দেওয়া; `finally`-তে পরিষ্কার; খালি `catch` কখনো নয়। এই অধ্যায় সেগুলো আবার শেখাবে না; বরং বড় ব্যবস্থায় সেগুলো কীভাবে একসঙ্গে কাজ করে তা দেখাবে।

১৬.২ Error-এর শ্রেণিবিন্যাস: code, retryable এবং cause

বড় Application-এ ত্রুটির একটি পরিবার থাকে। প্রতিটি সদস্যের নামের সঙ্গে একটি স্থির `code` (যা Log ও Server-এর সাড়ায় যায়, এবং ভাষা বদলালেও বদলায় না) এবং প্রয়োজনে `retryable` (আবার চেষ্টা করলে সফল হতে পারে কি না) রাখলে সামলানোর সিদ্ধান্ত সহজ হয়।

error-family.js · JavaScript — চলবে: Browser Console ও Node.js

```
class AppError extends Error {
  constructor(message, { code, retryable = false, cause } = {}) {
    super(message, { cause });
    this.name = new.target.name;
```

```
    this.code = code;
    this.retryable = retryable;
  }
}

class DatabaseError extends AppError {}
class SaveFailedError extends AppError {}

function causeChain(error) {
  const chain = [];
  for (let current = error; current; current = current.cause) {
    chain.push(`${current.name}: ${current.message}`);
  }
  return chain;
}

const low = new Error('connection reset');
const middle = new DatabaseError('INSERT ব্যর্থ', {
  code: 'DB_WRITE', retryable: true, cause: low,
});
const outer = new SaveFailedError('Booking সংরক্ষণ হয়নি', {
  code: 'SAVE', cause: middle,
});

console.log(causeChain(outer).join('\n  <- '));
console.log(outer instanceof AppError, outer.code, middle.retryable);
```

Output

```
SaveFailedError: Booking সংরক্ষণ হয়নি
  <- DatabaseError: INSERT ব্যর্থ
  <- Error: connection reset
true SAVE true
```

- `new.target.name` Constructor-কে যে Class দিয়ে ডাকা হয়েছে তার নাম দেয়। তাই প্রতিটি Child Class-এ আলাদা করে `this.name` লিখতে হয়নি (অধ্যায় ১৩-এর `this.name` সমস্যার স্থায়ী সমাধান)।
- **`cause`-এর শিকল** ধরে ওপর থেকে মূল কারণ পর্যন্ত যাওয়া যায়। ব্যবহারকারী দেখবেন শুধু ওপরের বার্তা; Log-এ যাবে পুরো শিকল।
- **ধরন দিয়ে পরিবার, `code` দিয়ে নির্দিষ্টতা:** `instanceof AppError` বলে এটি আমাদের প্রত্যাশিত ত্রুটি; `code` বলে ঠিক কোনটি।

একাধিক ত্রুটি একসঙ্গে ঘটলে JavaScript একটি **`AggregateError`** দেয়, যার `errors` Property-তে সবগুলো থাকে। যেমন `Promise.any`-এর সব Promise ব্যর্থ হলে:

aggregate.mjs · JavaScript চলবে: Browser Console ও Node.js

```
const fail = (ms, message) =>
  new Promise((_, reject) => setTimeout(() => reject(new Error(message)), ms));

try {
  await Promise.any([fail(10, 'Server A বন্ধ'), fail(20, 'Server B বন্ধ')]);
} catch (error) {
  console.log(error.name, error.errors.length);
  console.log(error.errors.map((item) => item.message).join(' | '));
}
```

Output

```
AggregateError 2
Server A বন্ধ | Server B বন্ধ
```

১৬.৩ Invariant: ভেতরের নিয়মের পাহারা

ইনপুট যাচাই বাইরের তথ্য পাহারা দেয় (Trust Boundary, অধ্যায় ১১)। কিন্তু Program-এর ভেতরেও কিছু নিয়ম আছে, যা কখনো ভাঙার কথা নয়: টাকা স্থানান্তরের পর মোট টাকা অপরিবর্তিত; আসনের সংখ্যা কখনো ঋণাত্মক নয়। এমন নিয়মকে বলে **Invariant**। ভাঙলে তা অবশ্যই একটি Bug, তাই সঙ্গে সঙ্গে জোরে ব্যর্থ হওয়াই ভালো (Fail Fast), ভুল অবস্থায় এগোনোর চেয়ে।

invariant.js · JavaScript চলবে: Browser Console ও Node.js

```
function invariant(condition, message) {
  if (!condition) {
    throw new Error(`Invariant ভঙ্গ: ${message}`);
  }
}

function transfer(accounts, from, to, amount) {
  invariant(Number.isInteger(amount) && amount > 0, 'পরিমাণ ধনাত্মক পূর্ণসংখ্যা');
  const totalBefore = accounts[from] + accounts[to];
  const next = {
    ...accounts,
    [from]: accounts[from] - amount,
    [to]: accounts[to] + amount,
  };

  invariant(next[from] >= 0, `${from}-এর Balance ঋণাত্মক হবে`);
  invariant(next[from] + next[to] === totalBefore, 'মোট টাকা অপরিবর্তিত');
  return next;
}

console.log(JSON.stringify(transfer({ a: 100, b: 50 }, 'a', 'b', 30)));
```

```
try {
  transfer({ a: 100, b: 50 }, 'a', 'b', 500);
} catch (error) {
  console.log(error.message);
}
```

Output

```
{"a":70,"b":80}
Invariant ভঙ্গ: a-এর Balance ঋণাত্মক হবে
```

শেষ Invariant ("মোট অপরিবর্তিত") এখন অপ্রয়োজনীয় মনে হতে পারে; কিন্তু ছয় মাস পরে কেউ Fee কাটার নিয়ম যোগ করলে এবং ভুল করে টাকা "হারিয়ে" ফেললে, এটিই প্রথম ধরবে। Node.js-এ একই কাজের জন্য Built-in `assert` Module আছে; Browser-এ `console.assert` শুধু বার্তা দেখায়, থামায় না।

১৬.৪ Asynchronous ত্রুটি এবং শেষ প্রতিরক্ষা

অধ্যায় ১০ ও ১১-এ Asynchronous ত্রুটি এবং Browser-এর `unhandledrejection` দেখেছেন। Node.js-এ একই শেষ প্রতিরক্ষা:

last-defense.js · JavaScript — চলবে: Node.js

```
process.on('unhandledRejection', (reason) => {
  console.log(`ধরা পড়েনি: ${reason.message}`);
});

async function saveReport() {
  throw new Error('Disk ভর্তি');
}

saveReport();
console.log('Program চলছে');
```

Output

```
Program চলছে
ধরা পড়েনি: Disk ভর্তি
```

Server-এ এখানে সাধারণত ত্রুটিটি Log করে Process-কে ব্যর্থ অবস্থায় বন্ধ করা হয় (`process.exitCode = 1`), যাতে একটি Process Manager সেটিকে পরিষ্কার অবস্থায় আবার চালু করে। অজানা অবস্থায় চলতে থাকা একটি Server বন্ধ হয়ে যাওয়া Server-এর চেয়ে বেশি বিপজ্জনক: সে ভুল তথ্য সংরক্ষণ করতে পারে।

১৬.৫ Debugging: একটি পুনরাবৃত্তিযোগ্য পদ্ধতি

অধ্যায় ১১-এ DevTools দেখেছেন। বড় Bug-এর জন্য হাতিয়ারের চেয়ে পদ্ধতি বেশি গুরুত্বপূর্ণ:

১. **নিশ্চিতভাবে আবার ঘটান।** কোন ইনপুট, কোন ক্রম, কোন পরিবেশ। যে Bug নিজের ইচ্ছায় ঘটানো যায় না, তা ঠিক হয়েছে কি না প্রমাণও করা যায় না।

২. **সবচেয়ে ছোট উদাহরণে নামান।** একটি একটি অংশ সরিয়ে দেখুন Bug থাকে কি না।

৩. **ধারণা লিখুন, তারপর প্রমাণ খুঁজুন।** "Payment দুবার কাটছে, কারণ Retry-র সময় আগের অনুরোধ আসলে সফল ছিল"—তারপর Log বা Breakpoint দিয়ে যাচাই।

৪. **অর্ধেক করে খুঁজুন (Binary Search)।** Program-এর মাঝখানে মান দেখুন: ঠিক থাকলে Bug পরের অর্ধেকে, নইলে আগের অর্ধেকে। কোন পরিবর্তনে Bug এসেছে তা খুঁজতে Git-এর `git bisect` একই কৌশল Commit-এর ওপর চালায়।

৫. **ঠিক করার আগে একটি ব্যর্থ Test লিখুন** (Regression Test), তারপর ঠিক করুন, তারপর Test পাস করছে দেখুন।

পর্যবেক্ষণের জন্য Log অপরিহার্য, বিশেষত যেখানে Debugger লাগানো যায় না (Server, ব্যবহারকারীর Browser)। মানুষের পড়ার বাক্যের চেয়ে **Structured Log**—প্রতিটি লাইন একটি JSON Object—অনেক বেশি কাজের, কারণ যন্ত্র দিয়ে খোঁজা ও গোনা যায়:

logger.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createLogger(context, write = console.log) {
  return (level, event, data = {}) =>
    write(JSON.stringify({ level, event, ...context, ...data }));
}

const log = createLogger({ service: 'booking' });

log('info', 'created', { id: 'B-1', seats: 2 });
log('warn', 'payment-retry', { attempt: 1 });
```

Output

```
{"level":"info","event":"created","service":"booking","id":"B-1","seats":2}
{"level":"warn","event":"payment-retry","service":"booking","attempt":1}
```

বাস্তব Log-এ সময় (`time`) এবং অনুরোধের ID-ও থাকে, যাতে একটি অনুরোধের সব লাইন একসঙ্গে খোঁজা যায়। **Log-এ কখনো গোপন তথ্য নয়:** Password, Card নম্বর, Token। `write` Inject করা যায়, তাই Test-এ Log-কে একটি Array-তে জমিয়ে যাচাই করা যায় (Project দেখুন)।

১৬.৬ Test-এর ধরন এবং Testing Pyramid

ধরন	কী যাচাই করে	গতি	উদাহরণ
Unit	একটি Function বা Class, বিচ্ছিন্নভাবে	খুব দ্রুত	`priceFor(5)`
Integration	কয়েকটি অংশ একসঙ্গে	দ্রুত	Service + Repository
End-to-End (E2E)	আসল Browser-এ ব্যবহারকারীর পুরো যাত্রা	ধীর	Form পূরণ থেকে নিশ্চিতকরণ
Regression	একটি পুরোনো Bug আর ফেরেনি	ধরনভেদে	দুবার Click-এ দুবার Payment নয়
Boundary	সীমার দুই পাশ	দ্রুত	০, ১, ১০, ১১ আসন

Testing Pyramid-এর পরামর্শ: নিচে অনেক Unit Test, মাঝে কিছু Integration Test, শীর্ষে অল্প কয়েকটি E2E Test। নিচের Test দ্রুত ও নিশ্চিত, এবং ব্যর্থ হলে ঠিক কোথায় সমস্যা তা দেখায়; ওপরের Test বাস্তবের সবচেয়ে কাছাকাছি, কিন্তু ধীর ও ভঙ্গুর। এই বইয়ের অধ্যায় ৯–১২ ও ১৫-এর `browser-check.mjs` E2E ধরনের; প্রতিটি Project-এর `node --test` Unit ও Integration।

Boundary Test সবচেয়ে বেশি Bug ধরে, কারণ ভুল প্রায়ই সীমায় থাকে: `<` না `<=`। "আসন ১ থেকে ১০" নিয়মের জন্য ০, ১, ১০ ও ১১—চারটি মান যাচাই যথেষ্ট; ৫ বা ৭-এর মতো মাঝের মান নতুন কিছু প্রমাণ করে না।

১৬.৭ node:test গভীরভাবে

এ পর্যন্ত আপনি `test()` ও `assert` ব্যবহার করেছেন। `node:test`-এ আরও আছে:

price.test.mjs · JavaScript — চলবে: Node.js

```
import { describe, it, beforeEach } from 'node:test';
import assert from 'node:assert/strict';

const SEAT_PRICE = 350;
const priceFor = (seats) => {
  const subtotal = seats * SEAT_PRICE;
  return seats >= 5 ? Math.round(subtotal * 0.9) : subtotal;
};

describe('priceFor', () => {
  let seatCounts;

  beforeEach(() => {
    seatCounts = [];
  });

  it('8 আসনে ছাড় নেই', () => {
    assert.equal(priceFor(4), 1400);
```

```
  });

  it('৫ আসন থেকে ১০% গ্রুপ ছাড়', () => {
    assert.equal(priceFor(5), 1575);
  });

  it('প্রতিটি Test নতুন অবস্থায় শুরু হয়', () => {
    seatCounts.push(1);
    assert.deepEqual(seatCounts, [1]);
  });

  it.todo('আসনের সর্বোচ্চ সীমা');
});
```

Terminal

```
node --test "examples/tests/*.test.mjs"
```

- **`describe`** ও **`it`** Test-গুলোকে দলে সাজায়; Report-এ দলের নাম দেখা যায়। `test()` ও `it()` একই জিনিস।
- **Hook**: `beforeEach` প্রতিটি Test-এর আগে চলে, তাই প্রতিটি Test নতুন, পরিষ্কার অবস্থায় শুরু হয়। Test পরস্পরের ওপর নির্ভর করলে একটির ব্যর্থতা অন্যগুলোকে বিভ্রান্তভাবে ব্যর্থ করে। `before`, `after`, `afterEach`-ও আছে।
- **`it.todo`** লেখা বাকি Test-এর জায়গা রাখে; Report-এ "todo" দেখায়, ব্যর্থ হয় না। `it.skip` একটি Test সাময়িকভাবে বন্ধ রাখে, কিন্তু দীর্ঘদিন Skip রাখা একটি সতর্কসংকেত।
- **চালানো**: Argument ছাড়া `node --test` বর্তমান Folder ও তার ভেতরের সব `*.test.mjs` খুঁজে চালায়; নির্দিষ্ট File-গুলো চাইলে উদ্ধৃতিচিহ্নের ভেতরে একটি Glob Pattern দিন, যেমন ওপরের Command। (Node.js 22-এ Folder-এর নাম সরাসরি দিলে কাজ করে না।) একটি নির্দিষ্ট Test: `node --test --test-name-pattern="গ্রুপ"`। লেখার সময় প্রতিবার সংরক্ষণে আবার চালাতে: `node --test --watch`।

এই অধ্যায়ের সব উদাহরণ-Test (`examples/tests/` এবং সমাধানের Test) Companion Source Code-এ একসঙ্গে চালিয়ে যাচাই করা হয়েছে।

ব্যর্থ Assertion কী তথ্য দেয়, দেখুন:

assert-message.mjs · JavaScript চলবে: Node.js

```
import assert from 'node:assert/strict';

const priceFor = (seats) => seats * 350;

try {
  assert.equal(priceFor(5), 1575, 'পাঁচ আসনে গ্রুপ ছাড় প্রত্যাশিত');
} catch (error) {
  console.log(error.code, error.actual, error.expected);
  console.log(error.message);
}
```

Output

```
ERR_ASSERTION 1750 1575
পাঁচ আসনে গ্রুপ ছাড় প্রত্যাশিত

1750 !== 1575
```

`actual` ও `expected` দেখায় কী পাওয়া গেল আর কী আশা করা হয়েছিল, এবং তৃতীয় Argument-এর বার্তা বলে **কেন** তা আশা করা হয়েছিল। Node.js 22 নিজের বার্তার নিচে তুলনাটিও (`1750 !== 1575`) যোগ করে দেয়। ভালো Test-এর ব্যর্থতা নিজেই একটি Bug Report।

Coverage: কোন Code কোনো Test ছোঁয়নি

Terminal

```
node --test --experimental-test-coverage
```

এই Command প্রতিটি File-এর কত শতাংশ লাইন, শাখা (Branch) ও Function Test চলার সময় ব্যবহৃত হয়েছে তার একটি সারণি দেখায়, এবং কোন লাইনগুলো কখনো চলেনি তা উল্লেখ করে। Project-এ এটি চালিয়ে অচলিত লাইনগুলো দেখলে প্রায়ই একটি অপরীক্ষিত ত্রুটির পথ পাওয়া যায়। তবে ১০০% Coverage মানে Bug-মুক্ত নয়: একটি লাইন চলেছে, কিন্তু তার ফল যাচাই হয়েছে কি না Coverage জানে না। Coverage দেখায় কোথায় Test **নেই**, কোথায় যথেষ্ট আছে তা নয়।

১৬.৮ Test Double: Stub, Spy, Mock এবং Fake

একটি Booking Service আসল Payment Gateway-তে টাকা কাটে, আসল Database-এ লেখে, আসল Email পাঠায়। Test-এ এগুলো ব্যবহার করা ধীর, ব্যয়বহুল, এবং অনিশ্চিত। তাদের জায়গায় নিয়ন্ত্রিত বিকল্প দেওয়া হয়, যাদের সাধারণ নাম **Test Double** (সিনেমার Stunt Double-এর মতো)।

ধরন	কী করে	কখন
Stub	নির্দিষ্ট উত্তর দেয়	"Payment সফল" বা "Payment ব্যর্থ" পরিস্থিতি তৈরি
Spy	আসলটির মতো কাজ করে, এবং প্রতিটি ডাক লিখে রাখে	কতবার, কী Argument-এ ডাকা হলো
Mock	আগে থেকে ঠিক করা প্রত্যাশা যাচাই করে	"Email ঠিক একবার পাঠানো হয়েছে"
Fake	সরল কিন্তু কার্যকর বিকল্প বাস্তবায়ন	Database-এর বদলে Memory-র Repository

doubles.test.mjs · JavaScript — চলবে: Node.js

```
import { test, mock } from 'node:test';
import assert from 'node:assert/strict';

async function placeOrder(order, { payment, mailer }) {
  const receipt = await payment.charge(order.amount);
  await mailer.send(order.email, `রসিদ ${receipt}`);
  return receipt;
}

test('Stub ও Mock: Payment-এর নির্দিষ্ট উত্তর, Email ঠিক একবার', async () => {
  const payment = { charge: async () => 'PAY-1' };
  const mailer = { send: mock.fn(async () => {}) };

  const order = { amount: 700, email: 'a@b.co' };
  const receipt = await placeOrder(order, { payment, mailer });

  assert.equal(receipt, 'PAY-1');
  assert.equal(mailer.send.mock.callCount(), 1);
  assert.deepEqual(mailer.send.mock.calls[0].arguments, ['a@b.co', 'রসিদ PAY-1']);
});

test('Stub-এর ব্যর্থতা: Payment ব্যর্থ হলে Email যায় না', async () => {
  const payment = { charge: async () => { throw new Error('declined'); } };
  const mailer = { send: mock.fn() };

  const order = { amount: 700, email: 'a@b.co' };
  await assert.rejects(placeOrder(order, { payment, mailer }), /declined/);
  assert.equal(mailer.send.mock.callCount(), 0);
});

test('Spy: একটি বিদ্যমান Object-এর Method-এর ডাক গোনা', () => {
  const calculator = { add: (a, b) => a + b };
  const spy = mock.method(calculator, 'add');

  assert.equal(calculator.add(2, 3), 5);
  assert.equal(spy.mock.callCount(), 1);
```

```
  spy.mock.restore();
});
```

- `mock.fn(impl)` একটি Function দেয় যা `impl`-এর মতো কাজ করে এবং প্রতিটি ডাক `mock.calls`-এ রাখে।
- `mock.method(object, 'name')` একটি বিদ্যমান Method-কে Spy-তে মোড়ায়; কাজ শেষে `restore()` আসলটি ফিরিয়ে দেয়।
- **নির্ভরতা Parameter হিসেবে** (`{ payment, mailer }`): অধ্যায় ১২-এর Dependency Injection ছাড়া এর কোনোটিই সম্ভব হতো না। Test-যোগ্য Code লেখার প্রথম শর্ত বাইরের জিনিস ভেতরে তৈরি না করে বাইরে থেকে নেওয়া।
- **সময়ও একটি নির্ভরতা।** `new Date()` সরাসরি ব্যবহার করলে Test-এর ফল তারিখের ওপর নির্ভর করে; একটি `clock` Function Inject করুন, অথবা অধ্যায় ১৪-এর Fake Timer।

সতর্কতা: Test Double যত বেশি, Test তত কম বাস্তবের সঙ্গে মেলে। প্রতিটি Mock একটি অনুমান: "আসল Payment Gateway এভাবেই আচরণ করে।" অনুমান ভুল হলে Test পাস করবে, Production ব্যর্থ হবে। তাই Fake-কে আসলের মতো আচরণ করান, এবং অন্তত কয়েকটি Integration বা E2E Test আসল অংশ দিয়ে চালান।

১৬.৯ ভালো Test-এর বৈশিষ্ট্য

- **একটি Test, একটি আচরণ;** নামেই সেই আচরণ: "Payment ব্যর্থ হলে সংরক্ষিত আসন ফেরত যায়"।
- **Arrange, Act, Assert:** প্রস্তুতি, একটি কাজ, যাচাই—তিনটি স্পষ্ট অংশ।
- **নিশ্চিত (Deterministic):** একই Code-এ সব সময় একই ফল। সময়, Random, Network—সব নিয়ন্ত্রিত। মাঝে মাঝে ব্যর্থ হওয়া (Flaky) Test-কে সবাই উপেক্ষা করতে শেখে, এবং তখন সেটি আসল Bug-ও ধরে না।
- **স্বাধীন:** কোনো Test অন্য Test-এর রেখে যাওয়া অবস্থার ওপর নির্ভর করে না; যেকোনো ক্রমে চলে।
- **আচরণ যাচাই করে, ভেতরের গঠন নয়:** Private Function-এর নাম বদলালে Test ভাঙা উচিত নয়, আচরণ বদলালে উচিত।
- **ব্যর্থতার পথও:** প্রতিটি `throw` এবং প্রতিটি `catch`-এর জন্য অন্তত একটি Test।
- **প্রতিটি Bug-এর জন্য একটি Regression Test,** যা আগের Code-এ ব্যর্থ হতো।

১৬.১০ সাধারণ ভুল

ভুল	সমস্যা	সমাধান
খালি `catch`	ত্রুটি অদৃশ্য	ধরুন যাচাই, অচেনা হলে আবার `throw`
সব ত্রুটিতে Retry	`400`-এ অকারণ চেষ্টা; দ্বিগুণ কাজ	শুধু `retryable`
Retry-তে Idempotency নেই	দুবার Payment	Idempotency Key

ভুল	সমস্যা	সমাধান
আংশিক ব্যর্থতায় ফেরত নেই	আসন বা টাকা "হারিয়ে" যায়	Compensation (ফেরত দেওয়া)
অপ্রধান কাজের ব্যর্থতায় পুরো কাজ ব্যর্থ	Email না যাওয়ায় Booking বাতিল	অপ্রধান কাজ আলাদা `try`, Log
Log-এ গোপন তথ্য	তথ্য ফাঁস	শুধু প্রয়োজনীয়, নিরাপদ Field
`new Date()`, `Math.random()` সরাসরি	Flaky Test	Inject করুন
Test-এ আসল Network	ধীর, অনিশ্চিত	Stub, Fake, Local Server
একটি Test-এ অনেক আচরণ	কোনটি ভাঙল বোঝা যায় না	একটি Test, একটি আচরণ
শুধু সফল পথের Test	ত্রুটির পথ অপরীক্ষিত	প্রতিটি `throw`-এর Test

১৬.১১ Debugging নির্দেশিকা

একটি Test ব্যর্থ হলে:

- **প্রথমে ব্যর্থতার বার্তা পড়ুন:** কোন Test, `actual`, `expected`, এবং Stack-এর প্রথম নিজস্ব লাইন।
- **শুধু সেই Test চালান:** `node --test --test-name-pattern="..."`; দ্রুত পুনরাবৃত্তি।
- **Test ভুল, না Code ভুল?** প্রত্যাশিত মানটি চাহিদা থেকে আবার হিসাব করুন। প্রায়ই Test-এর প্রত্যাশাই ভুল।
- **মাঝে মাঝে ব্যর্থ হয়?** সময়, Random, ক্রম, বা Test-এর মধ্যে ভাগ করা অবস্থা খুঁজুন; `beforeEach`-এ নতুন অবস্থা তৈরি করুন।
- **Mock-এর ডাক মিলছে না?** `mock.calls`-এর পুরো তালিকা Log করুন; প্রায়ই Argument-এর ক্রম বা ধরন আলাদা।
- **Test Debugger-এ:** `node --inspect-brk --test file.test.mjs` চালিয়ে Chrome-এর `chrome://inspect` থেকে Breakpoint দেওয়া যায়।

১৬.১২ পেশাদার অভ্যাস

- **প্রতিটি বাইরের নির্ভরতা Inject করুন:** Payment, Storage, Network, Clock, Logger।
- **ত্রুটির পরিবার, স্থির `code` এবং `retryable`।**
- **Retry শুধু সাময়িক ত্রুটিতে, সীমিত, Backoff-সহ, এবং Idempotency Key দিয়ে।**
- **বহু-ধাপের কাজে প্রতিটি ধাপের ব্যর্থতার জন্য ফেরত দেওয়ার পথ** (Compensation)।
- **প্রধান ও অপ্রধান কাজ আলাদা:** Booking সফল হলে Email-এর ব্যর্থতা শুধু Log।
- **Structured Log, গোপন তথ্য ছাড়া।**
- **Testing Pyramid:** অনেক দ্রুত Unit Test, কিছু Integration, অল্প E2E।

- **Bug মানে প্রথমে একটি ব্যর্থ Test, তারপর সংশোধন।**
- **Continuous Integration:** প্রতিটি পরিবর্তনে সব Test স্বয়ংক্রিয়ভাবে চালানো (GitHub Actions-এর মতো সেবা); লাল হলে Merge নয়।

১৬.১৩ Project: BookingGuard

একটি অনুষ্ঠানের আসন Booking Service। আসন সীমিত; প্রতিটি Booking-এ Payment কাটা হয় এবং Email-এ নিশ্চিতকরণ যায়। লক্ষ্য নির্ভরযোগ্যতা: Payment-এর সাময়িক ত্রুটিতে Retry, স্থায়ী ব্যর্থতায় সংরক্ষিত আসন ফেরত, দুবার পাঠানো অনুরোধে একবারই Payment, Email ব্যর্থ হলেও Booking অক্ষত, এবং সব ঘটনার Structured Log। প্রতিটি আচরণ Test Double দিয়ে প্রমাণিত।

Project-এর চাহিদা

- নাম কমপক্ষে ২ অক্ষর, Email-এর গঠন, আসন ১ থেকে ১০; সব ভুল একসঙ্গে (`ValidationError`)।
- আসনপ্রতি ৩৫০ টাকা; ৫ বা বেশি আসনে ১০% গ্রুপ ছাড়।
- আসন না থাকলে `SoldOutError` , এবং Payment-এর চেষ্টাই হবে না।
- Payment-এর সাময়িক ত্রুটিতে ২ বার পর্যন্ত Retry (১০০ ও ২০০ মিলিসেকেন্ড বিরতি); স্থায়ী বা শেষ ব্যর্থতায় আসন ফেরত।
- একই অনুরোধ-ID আবার এলে আগের Booking ফেরত; Payment আবার নয়।
- Email ব্যর্থ হলে Booking সফল থাকে; একটি `warn` Log।
- বাতিলে টাকা ফেরত (Refund) এবং আসন ফেরত; দুবার বাতিল নয়।

File-এর কাঠামো

Folder-এর কাঠামো

```
booking-guard/
├── src/
│   ├── errors.mjs              AppError পরিবার
│   ├── booking-core.mjs        যাচাই ও দাম (Pure)
│   ├── booking-service.mjs     Booking-এর প্রবাহ, Retry, Compensation
│   └── memory-repository.mjs   Database-এর Fake
├── demo.mjs
├── README.md
└── tests/
    ├── core.test.mjs
    ├── errors.test.mjs
    └── service.test.mjs
```

src/errors.mjs ও src/booking-core.mjs

errors.mjs · JavaScript চলবে: Browser Console ও Node.js

```
export class AppError extends Error {
  constructor(message, { code, retryable = false, cause } = {}) {
    super(message, { cause });
    this.name = new.target.name;
    this.code = code;
    this.retryable = retryable;
  }
}

export class ValidationError extends AppError {
  constructor(fields) {
    super('Booking-এর তথ্য ঠিক নেই।', { code: 'VALIDATION' });
    this.fields = fields;
  }
}

export class SoldOutError extends AppError {
  constructor(left) {
    super(`যথেষ্ট আসন নেই; বাকি ${left}টি।`, { code: 'SOLD_OUT' });
    this.left = left;
  }
}

export class PaymentError extends AppError {
  constructor(message, { retryable = false, cause } = {}) {
    super(message, { code: 'PAYMENT', retryable, cause });
  }
}
```

booking-core.mjs · JavaScript চলবে: Browser Console ও Node.js

```
import { ValidationError } from './errors.mjs';

export const MAX_SEATS = 10;
export const SEAT_PRICE = 350;
export const GROUP_SIZE = 5;
const EMAIL = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

export function validateRequest(raw) {
  const customer = String(raw?.customer ?? '').trim();
  const email = String(raw?.email ?? '').trim().toLowerCase();
  const seats = raw?.seats;
  const fields = {};

  if (customer.length < 2) {
    fields.customer = 'নাম কমপক্ষে 2 অক্ষরের হতে হবে।';
```

```
  }
  if (!EMAIL.test(email)) {
    fields.email = 'সঠিক একটি Email লিখুন।';
  }
  if (!Number.isInteger(seats) || seats < 1 || seats > MAX_SEATS) {
    fields.seats = `আসন 1 থেকে ${MAX_SEATS}-এর মধ্যে হতে হবে।`;
  }
  if (Object.keys(fields).length > 0) {
    throw new ValidationError(fields);
  }
  return { customer, email, seats };
}

export function priceFor(seats) {
  const subtotal = seats * SEAT_PRICE;
  return seats >= GROUP_SIZE ? Math.round(subtotal * 0.9) : subtotal;
}
```

src/memory-repository.mjs

memory-repository.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
import { SoldOutError } from './errors.mjs';

// Database-এর Fake: একই আচরণ, Memory-তে।
// আসল Database-এর Adapter-কেও এই চারটি Method দিতে হবে।
export function createMemoryRepository(capacity) {
  let available = capacity;
  const bookings = new Map();

  return {
    async reserve(seats) {
      if (seats > available) {
        throw new SoldOutError(available);
      }
      available -= seats;
    },
    async release(seats) {
      available += seats;
    },
    async save(booking) {
      bookings.set(booking.id, structuredClone(booking));
    },
    async find(id) {
      return bookings.has(id) ? structuredClone(bookings.get(id)) : null;
    },
    get available() {
      return available;
    },
```

```
  };
}
```

`reserve`-এ যাচাই ও বিয়োগের মাঝে কোনো `await` নেই, তাই একসঙ্গে দুটি অনুরোধ এলেও (অধ্যায় ১০) একটির যাচাই ও বিয়োগ শেষ হওয়ার আগে অন্যটি ঢুকতে পারে না: শেষ আসন দুজনকে দেওয়া হবে না। `structuredClone` (অধ্যায় ১২) নিশ্চিত করে যে বাইরের Code সংরক্ষিত Booking-কে সরাসরি বদলাতে পারে না, ঠিক আসল Database-এর মতো। Fake-এর এই আসলের-মতো আচরণই Test-কে বিশ্বাসযোগ্য করে।

src/booking-service.mjs

দুই অংশ মিলে সম্পূর্ণ File।

booking-service.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
import { priceFor, validateRequest } from './booking-core.mjs';
import { AppError, PaymentError } from './errors.mjs';

const realSleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

export function createBookingService({
  repository,
  payment,
  notifier,
  clock = () => new Date(),
  sleep = realSleep,
  log = () => {},
  retries = 2,
}) {
  async function chargeWithRetry(request, amount, requestId) {
    for (let attempt = 0; ; attempt += 1) {
      try {
        const order = { amount, email: request.email, key: requestId };
        return await payment.charge(order);
      } catch (error) {
        const retryable = error instanceof PaymentError && error.retryable;
        if (!retryable || attempt >= retries) {
          throw error;
        }
        log('warn', 'payment-retry', { requestId, attempt: attempt + 1 });
        await sleep(100 * 2 ** attempt);
      }
    }
  }

  async function notify(booking) {
    try {
      await notifier.send(booking.email, `Booking ${booking.id} নিশ্চিত`);
```

```
    } catch (error) {
      log('warn', 'notify-failed', { id: booking.id, reason: error.message });
    }
  }

  // ...
```

booking-service.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Browser Console ও Node.js

```
  // ...

  async function book(raw, requestId) {
    const existing = await repository.find(requestId);
    if (existing) {
      return existing;
    }

    const request = validateRequest(raw);
    const amount = priceFor(request.seats);
    await repository.reserve(request.seats);

    let receipt;
    try {
      receipt = await chargeWithRetry(request, amount, requestId);
    } catch (error) {
      await repository.release(request.seats);
      const code = error.code ?? 'UNKNOWN';
      log('error', 'payment-failed', { requestId, code });
      throw error;
    }

    const booking = {
      id: requestId, ...request, amount, receipt,
      status: 'confirmed', createdAt: clock().toISOString(),
    };
    await repository.save(booking);
    const { seats } = request;
    log('info', 'booking-confirmed', { id: requestId, seats, amount });
    await notify(booking);
    return booking;
  }

  async function cancel(id) {
    const booking = await repository.find(id);
    if (!booking) {
      throw new AppError(`${id} পাওয়া যায়নি।`, { code: 'NOT_FOUND' });
    }
    if (booking.status === 'cancelled') {
      throw new AppError(`${id} আগেই বাতিল।`, { code: 'ALREADY_CANCELLED' });
```

```
    }
    await payment.refund(booking.receipt);
    await repository.release(booking.seats);
    const cancelled = { ...booking, status: 'cancelled' };
    await repository.save(cancelled);
    log('info', 'booking-cancelled', { id });
    return cancelled;
  }

  return { book, cancel };
}
```

- **Factory ও Closure (অধ্যায় ১৪):** `createBookingService` সব নির্ভরতা একবার নেয়; ফেরত দেওয়া `book` ও `cancel` সেগুলো মনে রাখে। প্রতিটি নির্ভরতা Inject করা, এমনকি `sleep` ও `clock`।
- **ক্রম গুরুত্বপূর্ণ।** প্রথমে Idempotency যাচাই, তারপর ইনপুট যাচাই (সস্তা, কোনো Side Effect নেই), তারপর আসন সংরক্ষণ, তারপর Payment। আসন না থাকলে Payment-এর চেষ্টাই হয় না।
- **Retry** শুধু `retryable` `PaymentError`-এ, Backoff ১০০, ২০০; প্রতিটি Retry একটি `warn` Log। Payment-কে `key: requestId` পাঠানো হয়: আসল Payment Gateway এই **Idempotency Key** দেখে একই অনুরোধ দুবার এলে দ্বিতীয়বার টাকা কাটে না, যা Timeout-এর পর Retry-তে জরুরি (অধ্যায় ১০: সময়সীমা পার হওয়া মানে অনুরোধ ব্যর্থ নয়, শুধু উত্তর আসেনি)।
- **Compensation:** Payment চূড়ান্তভাবে ব্যর্থ হলে সংরক্ষিত আসন `release` হয়, তারপর ত্রুটি আবার ছোড়া হয়। কোনো আসন "আটকে" থাকে না।
- **প্রধান বনাম অপ্রধান:** Booking সংরক্ষণের পরে `notify` তার নিজের `try`-তে; Email ব্যর্থ হলে শুধু Log, Booking ফেরত যায়।
- `cancel` একই নীতি: প্রত্যাশিত ত্রুটির `code` (`NOT_FOUND`, `ALREADY_CANCELLED`), Refund, আসন ফেরত, নতুন অবস্থা সংরক্ষণ।

demo.mjs

demo.mjs · JavaScript — চলবে: Node.js

```
import { createBookingService } from './src/booking-service.mjs';
import { createMemoryRepository } from './src/memory-repository.mjs';
import { PaymentError } from './src/errors.mjs';

let charges = 0;
const payment = {
  async charge() {
    charges += 1;
    if (charges === 1) {
      throw new PaymentError('Gateway সাময়িক ব্যস্ত', { retryable: true });
    }
    return `PAY-${charges}`;
  },
```

```
  async refund() {},
};

const repository = createMemoryRepository(10);
const service = createBookingService({
  repository,
  payment,
  notifier: { send: async () => { throw new Error('SMTP বন্ধ'); } },
  clock: () => new Date('2026-12-01T10:00:00Z'),
  sleep: async () => {},
  log: (level, event, data) =>
    console.log(JSON.stringify({ level, event, ...data })),
});

const request = { customer: 'Sara', email: 'Sara@Example.com', seats: 5 };
const first = await service.book(request, 'REQ-1');
const again = await service.book(request, 'REQ-1');
const { available } = repository;
console.log(first.amount, first.receipt, again.receipt, charges, available);

const failing = [[{ ...request, seats: 6 }, 'REQ-2'], [{ seats: 0 }, 'REQ-3']];
for (const [raw, id] of failing) {
  try {
    await service.book(raw, id);
  } catch (error) {
    const detail = error.left ?? Object.keys(error.fields).join(',');
    console.log(error.name, error.code, detail);
  }
}
```

Output

```
{"level":"warn","event":"payment-retry","requestId":"REQ-1","attempt":1}
{"level":"info","event":"booking-confirmed","id":"REQ-1","seats":5,"amount":1575}
{"level":"warn","event":"notify-failed","id":"REQ-1","reason":"SMTP বন্ধ"}
1575 PAY-2 PAY-2 2 5
SoldOutError SOLD_OUT 5
ValidationError VALIDATION customer,email,seats
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **প্রথম Booking-এর তিনটি Log** পুরো গল্প বলে: Payment প্রথমবার ব্যর্থ, Retry-তে সফল; Booking নিশ্চিত; Email ব্যর্থ, কিন্তু Booking অক্ষত। ৫ আসনে গ্রুপ ছাড়: ১৭৫০-এর ১০% = ১৫৭৫।
- **দ্বিতীয়বার একই `REQ-1`**: একই রসিদ `PAY-2`, Payment-এর মোট ডাক এখনো ২ (একটি ব্যর্থ, একটি সফল), নতুন কোনো Log নেই। ব্যবহারকারী দুবার Click করলেও টাকা একবার।
- **বাকি ৫ আসন:** ৬ আসনের অনুরোধ `SoldOutError`, `left` ৫; Payment ডাকাই হয়নি।
- **অবৈধ অনুরোধ:** তিনটি ঘরের ভুল একসঙ্গে।

- `sleep: async () => {}`: প্রদর্শনীতে Backoff-এর আসল অপেক্ষা বাদ; Test এই `sleep`-কে Spy বানিয়ে যাচাই করে ঠিক ১০০ ও ২০০ মিলিসেকেন্ড চাওয়া হয়েছে কি না।
- **Email `Sara@Example.com` ছোট হাতের অক্ষরে** সংরক্ষিত, যাতে পরে খোঁজা ও তুলনা নির্ভরযোগ্য হয়।

চালানো এবং Test

Terminal

```
node demo.mjs
node --test
node --test --experimental-test-coverage
```

Companion Source Code-এর ২৪টি Test তিন স্তরে সাজানো:

File	স্তর	যা প্রমাণ করে
`core.test.mjs`	Unit, Boundary	০, ১, ১০, ১১ আসন; ৪ ও ৫ আসনের দাম; সব ভুল একসঙ্গে
`errors.test.mjs`	Unit	নাম, `code`, `retryable`, `cause`, `instanceof` শিকল
`service.test.mjs`	Integration (Fake Repository, Stub Payment, Mock Notifier, Spy Sleep)	Retry ও Backoff, Compensation, Sold Out-এ Payment নয়, Idempotency (Regression), Email ব্যর্থতা, বাতিল ও Refund, একসঙ্গে অনেক অনুরোধে আসনের সঠিক হিসাব

`--experimental-test-coverage` দিয়ে চালানোর আসল ফলের `src` অংশ (Node.js 22.16):

Terminal

```
# file                  | line % | branch % | funcs % | uncovered lines
#  booking-core.mjs     | 100.00 |   100.00 |  100.00 |
#  booking-service.mjs  | 100.00 |    96.15 |   62.50 |
#  errors.mjs           | 100.00 |   100.00 |  100.00 |
#  memory-repository.mjs | 100.00 |   100.00 |  100.00 |
```

`booking-service.mjs`-এ Function-এর Coverage ৬২.৫%, যদিও সব লাইন চলেছে। কারণটি শিক্ষণীয়: Default `sleep`, `clock` ও `log` Function কখনো চলেনি, কারণ প্রতিটি Test নিজের নিয়ন্ত্রিত বিকল্প দিয়েছে। এটি কোনো ফাঁক নয়, বরং Dependency Injection-এর প্রমাণ; আসল `sleep` ও `clock` পরীক্ষিত হয় শুধু Integration বা E2E স্তরে। Coverage-এর সংখ্যা তাই সব সময় পড়তে হয় তার কারণসহ।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Payment ব্যর্থতার পর আসন কমে থাকে	Compensation নেই	`catch`-এ `release`
দুবার Click-এ দুটি Booking	Idempotency যাচাই নেই	`find(requestId)` আগে
Email না যাওয়ায় Booking ব্যর্থ	`notify` আলাদা `try`-তে নেই	অপ্রধান কাজ আলাদা
Test ধীর (Backoff-এর অপেক্ষা)	আসল `sleep`	`sleep` Inject করুন
তারিখের ওপর নির্ভর করে Test ব্যর্থ	`new Date()` সরাসরি	`clock` Inject করুন
Retry হয় না	`retryable` মিথ্যা, বা `PaymentError` নয়	ত্রুটির ধরন ও Property

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। Repository Memory-তে, তাই Process বন্ধ হলে তথ্য যায়; আসল Database-এ "আসন সংরক্ষণ" একটি Transaction হতো, এবং একাধিক Server-এ চললে Memory-র `reserve` যথেষ্ট নয়। Payment ও Email কৃত্রিম; কোনো UI নেই। Refund ব্যর্থ হলে কী হবে (পুনরায় চেষ্টার একটি তালিকা) তা বাস্তবায়িত হয়নি।

১৬.১৪ অনুশীলনী

প্রাথমিক স্তর

১. প্রতিটি কোন ধরনের Test? (ক) `priceFor(5)` (খ) আসল Browser-এ Form পূরণ করে নিশ্চিতকরণ দেখা (গ) Service ও Fake Repository একসঙ্গে (ঘ) গত মাসের "দুবার Payment" Bug আর না ফেরা।

২. "বয়স ১৮ থেকে ৬০" নিয়মের Boundary মানগুলো লিখুন, এবং একটি Function দিয়ে যাচাই করুন।

৩. `isLeapYear(year)`-এর জন্য `node:test`-এ Test লিখুন: ২০২৪, ২০২৩, ১৯০০, ২০০০।

৪. একটি Test মাঝে মাঝে ব্যর্থ হয়: `assert.equal(greeting(), 'শুভ সকাল')`, যেখানে `greeting` `new Date().getHours()` দেখে। কেন? ঠিক করুন।

৫. `AppError` থেকে `NotFoundError` তৈরি করুন, যার `code` `NOT_FOUND`, এবং `name` আলাদা করে না লিখে।

মধ্যম স্তর

৬. `getTemperature(city, fetchImpl)`-এর Test লিখুন একটি Stub `fetch` দিয়ে: সফল ও `404`।

৭. `mock.fn` দিয়ে যাচাই করুন একটি `logger` ঠিক কোন Argument-এ ডাকা হয়েছে।

৮. ব্যবহারকারীদের জন্য একটি Memory-র Fake Repository লিখুন (`add`, `findByEmail`), এবং Test।

৯. একটি Bug: Coupon দুবার প্রয়োগ করলে ছাড় দুবার হয়। প্রথমে ব্যর্থ Regression Test, তারপর সংশোধন।

চ্যালেঞ্জ স্তর

১০. Inject করা `sleep` দিয়ে একটি Retry Function-এর Test লিখুন, যা প্রমাণ করে বিরতিগুলো ১০০, ২০০, ৪০০।

১১. তিনটি Service-এর Health Check `Promise.allSettled` দিয়ে চালিয়ে একটি Report দিন: কোনগুলো সুস্থ, কোনগুলো নয় এবং কেন।

১২. Project-এ একসঙ্গে ২০টি Booking অনুরোধ (প্রতিটি ১ আসন, মোট আসন ১০) পাঠিয়ে দেখান ঠিক ১০টি সফল এবং বাকি আসন ০।

১৬.১৫ অনুশীলনীর সমাধান

সমাধান ১ — (ক) Unit (খ) End-to-End (গ) Integration (ঘ) Regression। একটি Test একাধিক ধরনের হতে পারে: (ঘ) সাধারণত একটি Integration Test, যার উদ্দেশ্য Regression ধরা।

সমাধান ২

age-boundary.js · JavaScript — চলবে: Browser Console ও Node.js

```
const isEligible = (age) => Number.isInteger(age) && age >= 18 && age <= 60;

const boundaries = [17, 18, 60, 61];
console.log(boundaries.map((age) => `${age}:${isEligible(age)}`).join(' '));
```

Output

```
17:false 18:true 60:true 61:false
```

প্রতিটি সীমার দুই পাশে একটি করে মান; `>=` ভুল করে `>` লেখা হলে ১৮-এর Test সঙ্গে সঙ্গে ধরবে।

সমাধান ৩

leap-year.test.mjs · JavaScript — চলবে: Node.js

```
import { test } from 'node:test';
import assert from 'node:assert/strict';

const isLeapYear = (year) =>
  (year % 4 === 0 && year % 100 !== 0) || year % 400 === 0;

test('৪ দিয়ে বিভাজ্য: অধিবর্ষ', () => assert.equal(isLeapYear(2024), true));
test('৪ দিয়ে অবিভাজ্য: নয়', () => assert.equal(isLeapYear(2023), false));
test('১০০ দিয়ে বিভাজ্য, ৪০০ দিয়ে নয়: নয়', () =>
  assert.equal(isLeapYear(1900), false));
test('৪০০ দিয়ে বিভাজ্য: অধিবর্ষ', () => assert.equal(isLeapYear(2000), true));
```

চারটি Test নিয়মের চারটি শাখা ঢেকেছে; ১৯০০ ও ২০০০ হলো এই নিয়মের Boundary।

সমাধান ৪ — ফল নির্ভর করে Test কখন চলছে তার ওপর: সকালে পাস, বিকেলে ব্যর্থ। সময়কে Argument বানান:

greeting-clock.js · JavaScript — চলবে: Browser Console ও Node.js

```
function greeting(now = new Date()) {
  return now.getHours() < 12 ? 'শুভ সকাল' : 'শুভ অপরাহ্ন';
}

console.log(greeting(new Date(2026, 0, 1, 9, 0)));
console.log(greeting(new Date(2026, 0, 1, 15, 0)));
```

Output

```
শুভ সকাল
শুভ অপরাহ্ন
```

Production-এ Default আসল সময়; Test-এ নির্দিষ্ট সময়, তাই ফল সব সময় একই।

সমাধান ৫

not-found.js · JavaScript — চলবে: Browser Console ও Node.js

```
class AppError extends Error {
  constructor(message, { code } = {}) {
    super(message);
    this.name = new.target.name;
    this.code = code;
  }
}

class NotFoundError extends AppError {
  constructor(what) {
    super(`${what} পাওয়া যায়নি`, { code: 'NOT_FOUND' });
  }
}

const error = new NotFoundError('Booking B-9');
console.log(error.name, error.code, error.message, error instanceof AppError);
```

Output

```
NotFoundError NOT_FOUND Booking B-9 পাওয়া যায়নি true
```

সমাধান ৬

temperature.test.mjs · JavaScript — চলবে: Node.js

```
import { test } from 'node:test';
import assert from 'node:assert/strict';

async function getTemperature(city, fetchImpl = fetch) {
  const response = await fetchImpl(`/weather?city=${encodeURIComponent(city)}`);
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  return (await response.json()).celsius;
}

const stubFetch = (status, body) => async () =>
  new Response(JSON.stringify(body), { status });

test('সফল সাড়ায় তাপমাত্রা', async () => {
  const celsius = await getTemperature('Dhaka', stubFetch(200, { celsius: 31 }));
  assert.equal(celsius, 31);
});
```

```
test('404-এ ত্রুটি', async () => {
  const result = getTemperature('Atlantis', stubFetch(404, {}));
  await assert.rejects(result, { message: 'HTTP 404' });
});
```

সমাধান ৭

logger-mock.test.mjs · JavaScript — চলবে: Node.js

```
import { test, mock } from 'node:test';
import assert from 'node:assert/strict';

function register(user, logger) {
  logger('info', 'user-registered', { email: user.email });
  return { ...user, active: true };
}

test('নিবন্ধনে ঠিক একটি info Log, Password ছাড়া', () => {
  const logger = mock.fn();
  register({ email: 'a@b.co', password: 'secret' }, logger);

  assert.equal(logger.mock.callCount(), 1);
  const expected = ['info', 'user-registered', { email: 'a@b.co' }];
  assert.deepEqual(logger.mock.calls[0].arguments, expected);
  assert.doesNotMatch(JSON.stringify(logger.mock.calls), /secret/);
});
```

শেষ Assertion একটি নিরাপত্তা-Test: Log-এ Password কখনো যাবে না।

সমাধান ৮

user-repo.test.mjs · JavaScript — চলবে: Node.js

```
import { test } from 'node:test';
import assert from 'node:assert/strict';

function createUserRepository() {
  const users = new Map();
  return {
    async add(user) {
      const key = user.email.toLowerCase();
      if (users.has(key)) {
        throw new Error('Email আগে থেকেই আছে');
      }
      users.set(key, structuredClone(user));
    },
    async findByEmail(email) {
```

```
      const user = users.get(email.toLowerCase());
      return user ? structuredClone(user) : null;
    },
  };
}

test('যোগ ও খোঁজা, বড়-ছোট হাতের অক্ষর নির্বিশেষে', async () => {
  const repo = createUserRepository();
  await repo.add({ email: 'Sara@Example.com', name: 'Sara' });
  assert.equal((await repo.findByEmail('sara@example.com')).name, 'Sara');
  assert.equal(await repo.findByEmail('x@y.z'), null);
});

test('একই Email দুবার নয়; সংরক্ষিত Object অক্ষত', async () => {
  const repo = createUserRepository();
  await repo.add({ email: 'a@b.co', name: 'A' });
  await assert.rejects(repo.add({ email: 'A@B.CO', name: 'B' }), /আগে থেকেই/);
  const found = await repo.findByEmail('a@b.co');
  found.name = 'বদলানো';
  assert.equal((await repo.findByEmail('a@b.co')).name, 'A');
});
```

সমাধান ৯

coupon-regression.test.mjs · JavaScript — চলবে: Node.js

```
import { test } from 'node:test';
import assert from 'node:assert/strict';

// সংশোধিত সংস্করণ: Coupon আগে থেকে থাকলে আবার প্রয়োগ হয় না।
function applyCoupon(order, code) {
  if (order.coupons.includes(code)) {
    return order;
  }
  const coupons = [...order.coupons, code];
  return { ...order, coupons, total: order.total - 100 };
}

test('Regression: একই Coupon দুবার প্রয়োগে ছাড় একবার', () => {
  const order = { total: 1000, coupons: [] };
  const twice = applyCoupon(applyCoupon(order, 'SAVE100'), 'SAVE100');
  assert.equal(twice.total, 900);
});
```

পদ্ধতি: প্রথমে Test লিখুন পুরোনো Code-এর বিরুদ্ধে (সেখানে `total` হতো ৮০০, Test ব্যর্থ), তারপর `includes`-এর যাচাই যোগ করুন, Test পাস। Test-টি চিরকাল থাকবে, যাতে কেউ ভুল করে যাচাইটি মুছে দিলে সঙ্গে সঙ্গে ধরা পড়ে।

সমাধান ১০

retry-backoff.test.mjs · JavaScript চলবে: Node.js

```
import { test, mock } from 'node:test';
import assert from 'node:assert/strict';

async function withRetry(task, { retries = 3, sleep }) {
  for (let attempt = 0; ; attempt += 1) {
    try {
      return await task();
    } catch (error) {
      if (attempt >= retries) {
        throw error;
      }
      await sleep(100 * 2 ** attempt);
    }
  }
}

test('তিনবার ব্যর্থতার পর সফল: বিরতি 100, 200, 400', async () => {
  const sleep = mock.fn(async () => {});
  let calls = 0;
  const task = async () => {
    calls += 1;
    if (calls <= 3) {
      throw new Error('busy');
    }
    return 'ok';
  };

  assert.equal(await withRetry(task, { sleep }), 'ok');
  const waits = sleep.mock.calls.map((call) => call.arguments[0]);
  assert.deepEqual(waits, [100, 200, 400]);
});

test('সব চেষ্টা ব্যর্থ: শেষ ত্রুটি ছোড়ে, বিরতি তিনবার', async () => {
  const sleep = mock.fn(async () => {});
  const alwaysDown = async () => {
    throw new Error('down');
  };
  await assert.rejects(withRetry(alwaysDown, { sleep }), /down/);
  assert.equal(sleep.mock.callCount(), 3);
});
```

`sleep` Spy হওয়ায় Test আসলে এক মুহূর্তও অপেক্ষা করে না, অথচ প্রমাণ করে কতক্ষণ অপেক্ষা করা **হতো**।

সমাধান ১১

health.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const check = (name, ok) => (ok
  ? Promise.resolve(`${name} সুস্থ`)
  : Promise.reject(new Error(`${name} সাড়া দিচ্ছে না`)));

const results = await Promise.allSettled([
  check('Database', true),
  check('Payment', false),
  check('Email', true),
]);

const healthy = results.filter((result) => result.status === 'fulfilled');
console.log(`${healthy.length}/${results.length} সুস্থ`);

for (const result of results.filter((item) => item.status === 'rejected')) {
  console.log(`সমস্যা: ${result.reason.message}`);
}
```

Output

```
2/3 সুস্থ
সমস্যা: Payment সাড়া দিচ্ছে না
```

`Promise.all` প্রথম ব্যর্থতায় থেমে যেত এবং বাকি দুটির অবস্থা জানাত না; Health Check-এ প্রতিটির ফল দরকার।

সমাধান ১২

concurrent-booking.mjs · JavaScript — চলবে: Node.js

```
const SRC = '../../project/booking-guard/src';
const { createBookingService } = await import(`${SRC}/booking-service.mjs`);
const { createMemoryRepository } = await import(`${SRC}/memory-repository.mjs`);

const repository = createMemoryRepository(10);
const service = createBookingService({
  repository,
  payment: { charge: async () => 'PAY', refund: async () => {} },
  notifier: { send: async () => {} },
});

const requests = Array.from({ length: 20 }, (_, i) =>
  service.book({ customer: `User ${i}`, email: `u${i}@x.co`, seats: 1 }, i));
const results = await Promise.allSettled(requests);

const ok = results.filter((result) => result.status === 'fulfilled').length;
```

```
const soldOut = results.filter((r) => r.reason?.code === 'SOLD_OUT').length;
console.log(ok, soldOut, repository.available);
```

Output

```
10 10 0
```

২০টি অনুরোধ একসঙ্গে, প্রতিটি `await`-এ থেমে অন্যকে সুযোগ দিয়েছে; তবু ঠিক ১০টি সফল, বাকি ১০টি `SOLD_OUT`, এবং কোনো আসন ঋণাত্মক হয়নি। `reserve`-এর যাচাই ও বিয়োগের মাঝে `await` না থাকাই এর কারণ। Project-এর Test-ও এটি যাচাই করে।

১৬.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Error-এ `code` Property রাখার প্রধান সুবিধা কী?

ক) Error দ্রুত ছোড়া যায়

খ) বার্তার ভাষা বদলালেও Code ও Log-এ ত্রুটি চেনা যায়

গ) `stack` বড় হয়

ঘ) `catch` লাগে না

প্রশ্ন ২. `Promise.any`-এর সব Promise ব্যর্থ হলে কোন ধরনের ত্রুটি আসে?

ক) `TypeError`

খ) `RangeError`

গ) `AggregateError`

ঘ) `undefined`

প্রশ্ন ৩. Invariant ভাঙলে সাধারণত কী করা উচিত?

ক) চুপচাপ ঠিক করে এগোনো

খ) সঙ্গে সঙ্গে স্পষ্ট ত্রুটি দিয়ে থামা

গ) ব্যবহারকারীকে আবার চেষ্টা করতে বলা

ঘ) Log না করে উপেক্ষা

প্রশ্ন ৪. Testing Pyramid-এর নিচের স্তরে কোন Test সবচেয়ে বেশি?

ক) End-to-End

খ) Manual

গ) Unit

ঘ) Performance

প্রশ্ন ৫. "আসন ১ থেকে ১০" নিয়মের জন্য সবচেয়ে কার্যকর Test-মানের সেট?

ক) ৫

খ) ৩, ৫, ৭

গ) ১০০

ঘ) ০, ১, ১০, ১১

প্রশ্ন ৬. নির্দিষ্ট উত্তর দেয় এমন Test Double-এর নাম?

ক) Stub

খ) Fake

গ) Regression

ঘ) Coverage

প্রশ্ন ৭. Payment চূড়ান্তভাবে ব্যর্থ হলে সংরক্ষিত আসন ফেরত দেওয়াকে কী বলে?

ক) Retry

খ) Compensation

গ) Idempotency

ঘ) Memoization

প্রশ্ন ৮. ব্যবহারকারী দুবার Click করলেও একবারই Payment নিশ্চিত করার ধারণা?

ক) Throttle

খ) Coverage

গ) Idempotency

ঘ) Polymorphism

প্রশ্ন ৯. একটি Test মাঝে মাঝে ব্যর্থ হয়, কারণ তা `new Date()` সরাসরি ব্যবহার করে। সমাধান?

ক) Test আবার চালানো

খ) Test মুছে ফেলা

গ) Timeout বাড়ানো

ঘ) সময় (`clock`) Inject করা

প্রশ্ন ১০. ১০০% Coverage সম্পর্কে কোনটি সঠিক?

ক) Program Bug-মুক্ত

খ) প্রতিটি লাইন অন্তত একবার চলেছে, কিন্তু ফল যাচাই হয়েছে কি না বলে না

গ) আর Test লাগবে না

ঘ) E2E Test অপ্রয়োজনীয়

১৬.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–খ · ৪–গ · ৫–ঘ · ৬–ক · ৭–খ · ৮–গ · ৯–ঘ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	`code` স্থির; বার্তা মানুষের জন্য, `code` যন্ত্রের জন্য।
২	গ	`AggregateError`-এর `errors`-এ সব ত্রুটি থাকে।
৩	খ	Invariant ভাঙা মানে Bug; ভুল অবস্থায় এগোনো বিপজ্জনক।
৪	গ	Unit Test দ্রুত ও নিশ্চিত, তাই সবচেয়ে বেশি।
৫	ঘ	সীমার দুই পাশ; মাঝের মান নতুন কিছু প্রমাণ করে না।
৬	ক	Stub নির্দিষ্ট উত্তর দেয়; Fake কার্যকর বিকল্প বাস্তবায়ন।
৭	খ	আগের ধাপের প্রভাব উল্টে দেওয়া।
৮	গ	একই অনুরোধ বারবার এলেও ফল একবারের মতো।
৯	ঘ	সময় নিয়ন্ত্রণে এলে Test নিশ্চিত হয়।
১০	খ	Coverage দেখায় কোথায় Test নেই, Test যথেষ্ট কি না নয়।

১৬.১৮ অধ্যায়ের সারসংক্ষেপ

- Reliability-র চার দিক: সঠিকতা, সহনশীলতা, পুনরুদ্ধার, পর্যবেক্ষণযোগ্যতা।
- ত্রুটির পরিবার: ধরন, স্থির `code`, `retryable`, `cause`-এর শিকল; `new.target.name`; একাধিক ত্রুটিতে `AggregateError`।
- Invariant ভেতরের নিয়ম পাহারা দেয়; ভাঙলে Fail Fast।
- শেষ প্রতিরক্ষা: Browser-এ `unhandledrejection`, Node.js-এ `process.on('unhandledRejection')`।
- Debugging একটি পদ্ধতি: আবার ঘটানো, ছোট করা, ধারণা ও প্রমাণ, অর্ধেক করে খোঁজা, আগে Regression Test।
- Structured Log যন্ত্রে পাঠযোগ্য; গোপন তথ্য কখনো নয়।
- Testing Pyramid: অনেক Unit, কিছু Integration, অল্প E2E; Boundary-তে Bug বেশি।
- `node:test`: `describe`, `it`, Hook, `todo`, `skip`, Name Pattern, Watch, Coverage।
- Test Double: Stub, Spy, Mock, Fake; সব নির্ভরতা Inject করুন, সময়সহ।
- নির্ভরযোগ্য প্রবাহ: Retry শুধু সাময়িক ত্রুটিতে, Idempotency Key, ব্যর্থতায় Compensation, অপ্রধান কাজ আলাদা।

পরের অধ্যায়ে Regular Expression-এর গভীর ব্যবহার, উন্নত যাচাই এবং নিরাপদ Input Processing।

১৬.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Reliability	সব পরিস্থিতিতে Program-এর সঠিক ও প্রত্যাশিত আচরণ।
Error Code	ত্রুটির স্থির, যন্ত্রপাঠ্য পরিচয়।
AggregateError	একাধিক ত্রুটি একসঙ্গে বহনকারী Error।
Invariant	যে নিয়ম Program-এ সব সময় সত্য থাকার কথা।
Structured Log	যন্ত্রপাঠ্য গঠনের (যেমন JSON) Log লাইন।
Testing Pyramid	Unit, Integration, E2E-এর অনুপাতের নির্দেশিকা।
Boundary Test	সীমার দুই পাশের মান দিয়ে Test।
Regression Test	পুরোনো Bug না ফেরার Test।
Test Double	Test-এ আসল নির্ভরতার নিয়ন্ত্রিত বিকল্প।
Stub / Spy / Mock / Fake	নির্দিষ্ট উত্তর / ডাক লেখা / প্রত্যাশা যাচাই / কার্যকর বিকল্প।
Idempotency	একই অনুরোধ বারবার এলেও ফল একবারের মতো।
Compensation	ব্যর্থতার পর আগের ধাপের প্রভাব উল্টে দেওয়া।

১৬.২০ আরও পড়ুন

- **Node.js — Test runner:** `https://nodejs.org/api/test.html` — `describe`, Hook, Mock, Coverage।
- **Node.js — Assert:** `https://nodejs.org/api/assert.html` — সব Assertion।
- **MDN — AggregateError:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AggregateError` — একাধিক ত্রুটি।
- **MDN — new.target:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/new.target` — Constructor-এর ভেতরে ডাকার Class।
- **Martin Fowler — Test Double:** `https://martinfowler.com/bliki/TestDouble.html` — Stub, Mock, Fake-এর সংজ্ঞা।